

Functions, Arrays, Pointers & User Defined Data Types

Merged, topic-wise study notes: Units 3, 4 and 5 of
Fundamentals of Programming in C

What is inside

- Topic 1.** Functions: fundamentals and categories
- Topic 2.** Standard library functions
- Topic 3.** Arrays (1D, 2D, multi-dimensional, matrices)
- Topic 4.** Strings and string functions
- Topic 5.** Pointers and pointer arithmetic
- Topic 6.** Parameter passing and returning values
- Topic 7.** Recursion
- Topic 8.** Storage classes
- Topic 9.** Dynamic memory allocation
- Topic 10.** Structures
- Topic 11.** Union
- Topic 12.** Enumerated type (enum)
- Topic 13.** Quick revision and common mistakes

Every example program in this document was compiled and run, and the output shown is the real output.

Prepared by Brijesh Tankariya
S.V VIRANI COLLEGE

Contents

Topic 1: Functions — Fundamentals	4
1.1 Why Use Functions?	4
1.2 The Three Parts of a Function	4
1.3 Kinds of Functions	5
1.4 Four Categories of User-Defined Functions	5
1.5 The return Statement	6
Topic 2: Standard Library Functions	7
2.1 Mathematical Functions (math.h)	7
2.2 Input / Output Functions (stdio.h)	8
2.3 Character Functions (ctype.h)	8
2.4 General Utility Functions (stdlib.h)	9
Topic 3: Arrays	11
3.1 Key Characteristics	11
3.2 Single Dimensional Array	11
3.3 Two Dimensional Array	12
3.4 Multi-dimensional Array	15
3.5 Uses of Arrays	15
Topic 4: Strings	16
4.1 Declaring and Initialising Strings	16
4.2 Reading and Printing Strings	16
4.3 String Library Functions (string.h)	17
4.4 Writing Your Own String Functions	19
4.5 Array of Strings	20
Topic 5: Pointers	21
5.1 Address-of and Dereference Operators	21
5.2 Pointer to Array and Pointer Arithmetic	22
5.3 Pointers and Variable-Length Data: Array of Pointers	24
5.4 Pointer to Pointer (Double Pointer)	25
Topic 6: Parameter Passing and Returning Values	26
6.1 What Can Be Passed to a Function	26
6.2 Call by Value and Call by Reference	26
6.3 Passing Arrays to Functions	28
6.4 Returning Values	29
Topic 7: Recursion	30
7.1 The Two Parts of a Recursive Function	30
7.2 Factorial	30
7.3 More Recursion Examples	31
7.4 Recursion vs Iteration	32

Topic 8: Storage Classes	33
8.1 auto	33
8.2 register	33
8.3 static	34
8.4 extern	34
8.5 Summary of Storage Classes	35
Topic 9: Dynamic Memory Allocation	36
9.1 Stack vs Heap	36
9.2 Memory Allocation Functions (stdlib.h)	36
9.3 Dynamic Array with malloc	37
9.4 calloc and realloc	38
9.5 Dynamic Strings and Dynamic 2D Arrays	38
9.6 Memory Leak and the Dangling Pointer Problem	39
Topic 10: Structures	41
10.1 Declaring a Structure	41
10.2 Initialising and Accessing Members	41
10.3 Size, Copying and Comparing	42
10.4 Array of Structures	43
10.5 Pointers and Structures	44
10.6 Structures and Functions	45
10.7 Nested Structures	47
10.8 Dynamic Memory Allocation for Structures	48
10.9 typedef with Structures	49
Topic 11: Union	50
11.1 Declaring and Using a Union	50
11.2 Structure vs Union: Memory Comparison	50
11.3 Where Unions Are Useful	51
Topic 12: Enumerated Type (enum)	52
12.1 Default Numbering	52
12.2 Assigning Your Own Values	52
12.3 enum with switch	53
12.4 enum vs #define	53
12.5 Putting It Together: enum + struct + union (tagged union)	54
Topic 13: Quick Revision	55
13.1 Structure vs Union vs Enum at a Glance	55
13.2 Array vs Pointer	55
13.3 Key Points to Remember	55
13.4 Common Mistakes	56
13.5 Coverage Map: Where the Old Units Went	57

Topic 1: Functions — Fundamentals

A **function** is a self-contained block of code, given a name, that performs one specific, well-defined task. Instead of writing the same instructions again and again, the logic is written once and then **called** as many times as needed, from anywhere in the program. Every C program already uses at least one function: `main()`.

1.1 Why Use Functions?

- **Modularity:** a large program is split into small, independent pieces, each handling one task.
- **Reusability:** a function written once can be called many times (or reused in other programs).
- **Easier debugging and maintenance:** an error is traced to one small function; a change is made in one place.
- **Abstraction:** the caller only needs to know *what* a function does and how to call it, not *how* it works inside.
- **Shorter programs:** repeated code is replaced by a single call.

1.2 The Three Parts of a Function

Part	Purpose	Example
Declaration (Prototype)	Tells the compiler the function's name, return type and parameter types <i>before</i> it is used.	<code>int add(int a, int b);</code>
Definition	Contains the actual body: the statements that run when the function is called.	<code>int add(int a, int b) { return a + b; }</code>
Call	The statement that invokes the function, passing arguments.	<code>result = add(4, 6);</code>

```
#include <stdio.h>

int add(int a, int b);           /* 1. Declaration (prototype) */

int main() {
    int result = add(4, 6);      /* 3. Function call */
    printf("Sum = %d\n", result);
    return 0;
}

int add(int a, int b) {         /* 2. Definition (body) */
    return a + b;
}
```

Output:

Sum = 10

General syntax of a function definition:

```
return_type function_name(parameter_list)
{
    /* body: declarations and statements */
    return value;           /* omitted when return_type is void */
}
```

- **Formal parameters:** the variables listed in the function *definition* (`int a, int b`).
- **Actual arguments:** the values or variables written in the function *call* (`4, 6`). They map onto the formal parameters in order.

- If the function is defined **before** `main()`, a separate prototype is not required, but writing prototypes at the top is good practice.

1.3 Kinds of Functions

- **Library (inbuilt) functions:** pre-written and pre-compiled functions supplied with the compiler, such as `printf()`, `strlen()` and `sqrt()`. A header file (`<stdio.h>`, `<string.h>`, ...) must be included. These are covered in Topic 2 and Topic 4.
- **User-defined functions:** functions the programmer writes for the needs of a specific program.

1.4 Four Categories of User-Defined Functions

Every user-defined function can be classified by two independent questions: does it **accept arguments**, and does it **return a value**?

Category	Description	Example signature
No arguments, no return value	Performs a fixed task; typically prints a fixed message.	<code>void printMessage()</code>
No arguments, returns a value	Computes and returns a value without any input.	<code>int getYear()</code>
Arguments, no return value	Takes input, only produces a side effect such as printing.	<code>void printSum(int a, int b)</code>
Arguments, returns a value	Most flexible and most common: input in, result out.	<code>int multiply(int a, int b)</code>

```
#include <stdio.h>

void printMessage() {                /* 1. no arguments, no return value */
    printf("Welcome to C!\n");
}

int getYear() {                      /* 2. no arguments, returns a value */
    return 2024;
}

void printSum(int a, int b) {        /* 3. arguments, no return value */
    printf("Sum = %d\n", a + b);
}

int multiply(int a, int b) {         /* 4. arguments, returns a value */
    return a * b;
}

int main() {
    printMessage();
    printf("Year = %d\n", getYear());
    printSum(3, 4);
    printf("Product = %d\n", multiply(3, 4));
    return 0;
}
```

Output:

```
Welcome to C!
Year = 2024
Sum = 7
Product = 12
```

1.5 The return Statement

- A function sends **exactly one value** back to its caller using **return**. Its type must match (or be convertible to) the declared return type.
- If nothing needs to be returned, the return type is **void**; a bare **return**; may be used to leave the function early.
- **return** ends the function immediately: statements after it in the same path are never executed.
- To "return" several results, pass **pointers** as parameters (see Topic 6.4).

```
#include <stdio.h>

int isPositive(int n) {
    if (n > 0)
        return 1;      /* leaves the function here */
    return 0;
}

void greet(int hour) {
    if (hour < 12) {
        printf("Good morning\n");
        return;         /* early exit from a void function */
    }
    printf("Good day\n");
}

int main() {
    printf("isPositive(5) = %d\n", isPositive(5));
    printf("isPositive(-2) = %d\n", isPositive(-2));
    greet(9);
    greet(15);
    return 0;
}
```

Output:

```
isPositive(5) = 1
isPositive(-2) = 0
Good morning
Good day
```

Topic 2: Standard Library Functions

Library functions are ready-made functions written, tested and compiled by the compiler vendor. To use one, the program must **#include** the header file that contains its prototype.

Header	Provides	Typical functions
<code><stdio.h></code>	Standard input / output	<code>printf</code> , <code>scanf</code> , <code>gets</code> , <code>puts</code> , <code>getchar</code> , <code>putchar</code> , <code>getc</code> , <code>putc</code> , <code>ungetc</code>
<code><string.h></code>	String handling (see Topic 4)	<code>strlen</code> , <code>strcpy</code> , <code>strcat</code> , <code>strcmp</code> , <code>strchr</code> , <code>strstr</code> , <code>strtok</code>
<code><math.h></code>	Mathematical functions	<code>sqrt</code> , <code>pow</code> , <code>sin</code> , <code>cos</code> , <code>ceil</code> , <code>floor</code> , <code>fabs</code> , <code>log</code>
<code><ctype.h></code>	Character testing and conversion	<code>isalpha</code> , <code>isdigit</code> , <code>toupper</code> , <code>tolower</code>
<code><stdlib.h></code>	General utilities	<code>abs</code> , <code>atof</code> , <code>exit</code> , <code>rand</code> , <code>srand</code> , <code>malloc</code> , <code>free</code> (Topic 9)

2.1 Mathematical Functions (math.h)

Most **math.h** functions take and return **double**, whatever the type of the argument passed in.

Category	Functions	Description
Trigonometry	<code>sin(x)</code> , <code>cos(x)</code> , <code>tan(x)</code> , <code>asin(x)</code> , <code>acos(x)</code> , <code>atan(x)</code>	Standard and inverse trigonometric functions; angles are in radians .
Powers and roots	<code>pow(x, y)</code> , <code>sqrt(x)</code> , <code>exp(x)</code>	x raised to the power y; square root; e raised to the power x.
Rounding	<code>ceil(x)</code> , <code>floor(x)</code>	Round up / round down to the nearest integer value.
Absolute value	<code>fabs(x)</code>	Absolute value of a floating-point number.
Division and remainder	<code>fmod(x, y)</code> , <code>modf()</code> , <code>div()</code>	Floating-point remainder; split into integer and fractional parts; integer quotient and remainder together.
Logarithms	<code>log(x)</code> , <code>log10(x)</code>	Natural (base e) and base-10 logarithm.

```
#include <stdio.h>
#include <math.h>

int main() {
    double base = 2.0, exponent = 3.0;
    printf("pow(2, 3)    = %.2f\n", pow(base, exponent));
    printf("sqrt(16)     = %.2f\n", sqrt(16.0));
    printf("ceil(4.1)    = %.2f\n", ceil(4.1));
    printf("floor(4.9)   = %.2f\n", floor(4.9));
    printf("fabs(-7.5)   = %.2f\n", fabs(-7.5));
    printf("fmod(10, 3)  = %.2f\n", fmod(10.0, 3.0));
    printf("log10(1000) = %.2f\n", log10(1000.0));
    return 0;
}
```

Output:

```
pow(2, 3)    = 8.00
sqrt(16)     = 4.00
ceil(4.1)    = 5.00
floor(4.9)   = 4.00
fabs(-7.5)   = 7.50
fmod(10, 3)  = 1.00
log10(1000)  = 3.00
```

NOTE: On some platforms the math library must be linked explicitly, e.g. `gcc prog.c -lm` on Linux.

2.2 Input / Output Functions (stdio.h)

Function	Type	Description
<code>printf()</code> / <code>scanf()</code>	Formatted	Print formatted output / read formatted input using format specifiers <code>%d</code> , <code>%c</code> , <code>%s</code> , <code>%f</code> .
<code>gets()</code> / <code>puts()</code>	String	Read / print a line of text. <code>gets()</code> is unsafe (no bounds check) and removed in C11; use <code>fgets()</code> .
<code>getchar()</code> / <code>putchar()</code>	Character	Read / print a single character from / to the console.
<code>getc()</code> / <code>putc()</code>	Stream	Like <code>getchar</code> / <code>putchar</code> , but work on any stream, including files.
<code>ungetc()</code>	Character	Pushes a character back onto the input stream so it is read next.

```
#include <stdio.h>

int main() {
    int age;
    float height;
    char grade;

    scanf("%d %f %c", &age, &height, &grade); /* formatted input */
    printf("Age = %d, Height = %.1f, Grade = %c\n", age, height, grade);

    putchar('O');          /* single-character output */
    putchar('K');
    putchar('\n');
    puts("puts adds a newline automatically");
    return 0;
}
```

Sample input:

20 5.6 A

Output:

Age = 20, Height = 5.6, Grade = A
OK
puts adds a newline automatically

CAUTION: ****Why `gets()` is unsafe:**** it reads until a newline with no way to limit the length, so a long line writes past the end of the array. `fgets(buffer, size, stdin)` takes the buffer size as an argument and is safe (Topic 4.2).

2.3 Character Functions (ctype.h)

The testing functions return non-zero (true) if the condition holds and zero (false) otherwise.

Function	Meaning
<code>isalpha(c)</code> , <code>isdigit(c)</code> , <code>isalnum(c)</code>	Letter? Digit 0-9? Letter or digit?
<code>isupper(c)</code> , <code>islower(c)</code>	Uppercase letter? Lowercase letter?
<code>isspace(c)</code> , <code>isxdigit(c)</code>	Whitespace character? Valid hexadecimal digit?
<code>toupper(c)</code> , <code>tolower(c)</code>	Convert to uppercase / lowercase. Non-letters are returned unchanged.

```
#include <stdio.h>
#include <ctype.h>

int main() {
    char text[] = "Hello World 2024!";
    int upper = 0, lower = 0, digit = 0, space = 0;

    for (int i = 0; text[i] != '\0'; i++) {
        if (isupper(text[i])) upper++;
        else if (islower(text[i])) lower++;
        else if (isdigit(text[i])) digit++;
        else if (isspace(text[i])) space++;
    }
    printf("Upper=%d Lower=%d Digits=%d Spaces=%d\n", upper, lower, digit, space);

    for (int i = 0; text[i] != '\0'; i++)
        text[i] = toupper(text[i]);    /* convert every letter */
    printf("%s\n", text);
    return 0;
}
```

Output:

```
Upper=2 Lower=8 Digits=4 Spaces=2
HELLO WORLD 2024!
```

2.4 General Utility Functions (stdlib.h)

Category	Functions	Description
Math	<code>abs()</code> , <code>labs()</code>	Absolute value of an <code>int</code> / a <code>long</code> .
Conversion	<code>atof()</code> , <code>atol()</code> , <code>strtoul()</code>	String to <code>double</code> / <code>long</code> / <code>unsigned long</code> . <code>strtoul</code> reports where conversion stopped (more robust).
Program control	<code>exit()</code>	Terminates the program immediately, returning a status code to the operating system.
Randomness	<code>rand()</code> , <code>srand()</code>	<code>rand()</code> gives a pseudo-random integer; <code>srand()</code> seeds the generator (often with the current time).

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    printf("abs(-25)          = %d\n", abs(-25));
    printf("labs(-70000)     = %ld\n", labs(-70000L));
    printf("atof(\"3.14abc\")    = %.2f\n", atof("3.14abc"));
    printf("atol(\"123456\")+1    = %ld\n", atol("123456") + 1);

    char *stop;
    unsigned long v = strtoul("255xyz", &stop, 10);
    printf("strtoul -> %lu, stopped at \"%s\"\n", v, stop);

    exit(0);    /* terminate the program here */
    printf("This line never runs\n");
}
```

Output:

```
abs(-25)          = 25
labs(-70000)     = 70000
atof("3.14abc")  = 3.14
atol("123456")+1 = 123457
strtoul -> 255, stopped at "xyz"
```

Random numbers: `rand()` returns the same sequence on every run unless the generator is seeded once with a changing value such as the current time.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main() {
    srand(time(NULL));           /* seed once, at the start */
    int dice = rand() % 6 + 1;   /* value between 1 and 6 */
    printf("Dice roll: %d\n", dice);
    return 0;
}
```

Output:

Dice roll: 4 (a different value in 1..6 on each run)

NOTE: `delay()` and `clrscr()` belong to the old Turbo C `<conio.h>` / `<dos.h>` headers. They are **not** part of standard C and are obsolete on modern compilers. `errno` (declared in `<errno.h>`) is a global variable that library functions use to report the reason for a runtime error.

Topic 3: Arrays

An **array** is a collection of elements of the **same data type** stored in **contiguous** (back-to-back) memory locations. Instead of declaring **num1**, **num2**, **num3**, ... a set of related values is kept under one name, e.g. **num[3]**, and processed with a loop.

3.1 Key Characteristics

- All elements have the same type and therefore the same size.
- Contiguous storage allows very fast, direct access to any element through its **index**.
- Indexing is **zero-based**: the first element is **a[0]** and the last element of an array of size **n** is **a[n-1]**.
- The size of a normal (static) array must be known at compile time and cannot change afterwards.
- C does **not** check bounds. Accessing an index outside 0 to **n-1** (an *out-of-bounds access*) can silently corrupt other memory or crash the program.
- An array cannot be copied with **=**; elements must be copied one by one (or with **memcpy**).

3.2 Single Dimensional Array

A linear list of items; each element is reached with exactly one index.

```
int marks[5];           /* declaration: 5 uninitialised ints      */
int a[5] = {85, 90, 78, 66, 92}; /* full initialisation                          */
int b[5] = {1, 2};      /* partial: remaining elements become 0         */
int c[] = {4, 5, 6};    /* size omitted: compiler counts it as 3        */
```

Memory layout of **int a[5]** (assuming a 4-byte **int** and base address 1000):

Index	a[0]	a[1]	a[2]	a[3]	a[4]
Value	85	90	78	66	92
Address	1000	1004	1008	1012	1016

Example: input, sum and average. The number of elements is found with **sizeof(a) / sizeof(a[0])**.

```
#include <stdio.h>

int main() {
    int a[5], sum = 0;
    int n = sizeof(a) / sizeof(a[0]); /* 20 bytes / 4 bytes = 5 */

    for (int i = 0; i < n; i++) { /* reading into the array */
        scanf("%d", &a[i]);
        sum += a[i];
    }
    printf("Elements: ");
    for (int i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\nSum = %d, Average = %.2f\n", sum, (float)sum / n);
    return 0;
}
```

Sample input:

```
10 20 30 40 50
```

Output:

```
Elements: 10 20 30 40 50
Sum = 150, Average = 30.00
```

Example: largest and smallest element (a very common pattern: assume the first element is both, then compare).

```
#include <stdio.h>

int main() {
    int a[6] = {45, 12, 89, 33, 7, 61};
    int max = a[0], min = a[0];

    for (int i = 1; i < 6; i++) {
        if (a[i] > max) max = a[i];
        if (a[i] < min) min = a[i];
    }
    printf("Max = %d, Min = %d\n", max, min);
    return 0;
}
```

Output:

Max = 89, Min = 7

Example: linear search and bubble sort (sorting and searching are the classic array algorithms).

```
#include <stdio.h>

int main() {
    int a[5] = {64, 25, 12, 22, 11};
    int n = 5, key = 22, found = -1;

    /* Linear search */
    for (int i = 0; i < n; i++) {
        if (a[i] == key) { found = i; break; }
    }
    if (found >= 0) printf("%d found at index %d\n", key, found);
    else printf("%d not found\n", key);

    /* Bubble sort (ascending): repeatedly swap neighbours that are out of order */
    for (int pass = 0; pass < n - 1; pass++) {
        for (int j = 0; j < n - 1 - pass; j++) {
            if (a[j] > a[j + 1]) {
                int temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }
    printf("Sorted: ");
    for (int i = 0; i < n; i++) printf("%d ", a[i]);
    printf("\n");
    return 0;
}
```

Output:

22 found at index 3
Sorted: 11 12 22 25 64

3.3 Two Dimensional Array

A 2D array is an "array of arrays", used for tables, grids and **matrices**. Two indices are needed:

a[row][column].

```
int matrix[2][3] = { {1, 2, 3},          /* row 0 */
                    {4, 5, 6} };        /* row 1 */
```

Memory layout: C stores a 2D array in **row-major order**. All of row 0 comes first, immediately followed by row 1, and so on. So `matrix[2][3]` sits in memory as 1 2 3 4 5 6.

```
#include <stdio.h>

int main() {
    int matrix[2][3] = { {1, 2, 3}, {4, 5, 6} };

    printf("Value at row 1, col 2: %d\n\n", matrix[1][2]);

    for (int i = 0; i < 2; i++) {           /* outer loop: rows */
        for (int j = 0; j < 3; j++) {       /* inner loop: columns */
            printf("%3d", matrix[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

Output:

Value at row 1, col 2: 6

```
1  2  3
4  5  6
```

Matrix Operations Using 2D Arrays

Addition: corresponding elements are added. Both matrices must have the same size.

```
#include <stdio.h>

int main() {
    int a[2][2] = { {1, 2}, {3, 4} };
    int b[2][2] = { {5, 6}, {7, 8} };
    int sum[2][2];

    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            sum[i][j] = a[i][j] + b[i][j];
            printf("%d ", sum[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

Output:

```
6 8
10 12
```

Multiplication: an (m x n) matrix can be multiplied by an (n x p) matrix. Each result element is the sum of products of one **row** of the first matrix and one **column** of the second, which needs a **third** nested loop.

```
#include <stdio.h>

int main() {
    int a[2][3] = { {1, 2, 3}, {4, 5, 6} }; /* 2 x 3 */
    int b[3][2] = { {7, 8}, {9, 10}, {11, 12} }; /* 3 x 2 */
    int c[2][2]; /* result: 2 x 2 */

    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            c[i][j] = 0;
            for (int k = 0; k < 3; k++)
                c[i][j] += a[i][k] * b[k][j]; /* row i of a x column j of b */
        }
    }
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++)
            printf("%4d", c[i][j]);
        printf("\n");
    }
    return 0;
}
```

Output:

```
58  64
139 154
```

Transpose: rows become columns, so $t[j][i] = a[i][j]$.

```
#include <stdio.h>

int main() {
    int a[2][3] = { {1, 2, 3}, {4, 5, 6} };
    int t[3][2];

    for (int i = 0; i < 2; i++)
        for (int j = 0; j < 3; j++)
            t[j][i] = a[i][j];

    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 2; j++)
            printf("%d ", t[i][j]);
        printf("\n");
    }
    return 0;
}
```

Output:

```
1 4
2 5
3 6
```

3.4 Multi-dimensional Array

C supports three or more dimensions. A 3D array is an array of 2D arrays: a stack of tables. The total number of elements is the **product of all dimension sizes**: `int cube[2][3][4]` holds $2 \times 3 \times 4 = 24$ integers.

```
#include <stdio.h>

int main() {
    /* 2 blocks, each with 2 rows and 3 columns */
    int cube[2][2][3] = { { {1, 2, 3}, {4, 5, 6} },
                          { {7, 8, 9}, {10, 11, 12} } };
    int total = 0;

    for (int b = 0; b < 2; b++)
        for (int r = 0; r < 2; r++)
            for (int c = 0; c < 3; c++)
                total += cube[b][r][c];

    printf("cube[1][0][2] = %d\n", cube[1][0][2]);
    printf("Elements = %zu, Sum = %d\n", sizeof(cube) / sizeof(int), total);
    return 0;
}
```

Output:

```
cube[1][0][2] = 9
Elements = 12, Sum = 78
```

3.5 Uses of Arrays

- **Data storage:** many related values under one variable name.
- **Cleaner code:** loops replace many near-identical statements.
- **Building block for data structures:** stacks, queues, hash tables.
- **Sorting and searching:** bubble sort, binary search, linear search.
- **Matrices and tables:** 2D arrays map directly onto rows and columns.

TIP: Arrays are passed to functions as pointers (Topic 6.3) and are closely tied to pointer arithmetic (Topic 5.2). Character arrays are the basis of strings (Topic 4).

Topic 4: Strings

C has no separate string data type. A **string** is a one-dimensional array of **char** ending with the special **null character** `'\0'` (ASCII value 0). The terminator marks where the string ends; `strlen()`, `printf("%s")` and every other string function rely on it.

4.1 Declaring and Initialising Strings

```
#include <stdio.h>
#include <string.h>

int main() {
    char s1[] = "Hello";           /* compiler adds '\0' */
    char s2[6] = {'H', 'e', 'l', 'l', 'o', '\0'}; /* same thing, by hand */
    char s3[20] = "Hello";        /* extra space is fine */

    printf("s1 = %s, sizeof = %zu, strlen = %zu\n", s1, sizeof(s1), strlen(s1));
    printf("s2 = %s, sizeof = %zu, strlen = %zu\n", s2, sizeof(s2), strlen(s2));
    printf("s3 = %s, sizeof = %zu, strlen = %zu\n", s3, sizeof(s3), strlen(s3));
    return 0;
}
```

Output:

```
s1 = Hello, sizeof = 6, strlen = 5
s2 = Hello, sizeof = 6, strlen = 5
s3 = Hello, sizeof = 20, strlen = 5
```

sizeof gives the size of the whole array (including `'\0'` and unused space), while **strlen** counts characters **before** the null terminator. In memory, `"Hello"` is stored as:

Index	0	1	2	3	4	5
Character	'H'	'e'	'l'	'l'	'o'	'\0'

CAUTION: Because a string is an array, it cannot be assigned with `=` after declaration (`s1 = "Bye";` is an error). Use `strcpy(s1, "Bye")` instead. Also, `char *p = "Hello";` points to a read-only literal that must not be modified.

4.2 Reading and Printing Strings

Method	Behaviour
<code>scanf("%s", str)</code>	Stops at the first whitespace, so it reads only one word . No <code>&</code> is needed because the array name is already an address.
<code>fgets(str, size, stdin)</code>	Reads a whole line (spaces included) and never writes more than size characters. Keeps the trailing <code>'\n'</code> . Recommended .
<code>gets(str)</code>	Reads a whole line but has no size limit. Unsafe; removed in C11 .
<code>printf("%s", str) / puts(str)</code>	Print a string. puts adds a newline automatically.

```
#include <stdio.h>

int main() {
    char word[30];
    scanf("%s", word);          /* reads only up to the first space */
    printf("scanf read: %s\n", word);
    return 0;
}
```

Sample input:

Riya Shah

Output:

scanf read: Riya

```
#include <stdio.h>
#include <string.h>

int main() {
    char name[30];
    fgets(name, sizeof(name), stdin);    /* reads the whole line safely */
    name[strcspn(name, "\n")] = '\0';    /* remove the trailing newline */
    printf("Hello, %s!\n", name);
    return 0;
}
```

Sample input:

Riya Shah

Output:

Hello, Riya Shah!

4.3 String Library Functions (string.h)

Function	Description
<code>strlen(s)</code>	Length of <code>s</code> , not counting the null terminator.
<code>strcpy(s1, s2)</code> / <code>strncpy(s1, s2, n)</code>	Copy <code>s2</code> into <code>s1</code> . <code>strncpy</code> copies at most <code>n</code> characters.
<code>strcat(s1, s2)</code> / <code>strncat(s1, s2, n)</code>	Append <code>s2</code> to the end of <code>s1</code> . <code>strncat</code> appends at most <code>n</code> characters.
<code>strcmp(s1, s2)</code> / <code>strncmp(s1, s2, n)</code>	Compare character by character. Returns 0 if equal, negative if <code>s1 < s2</code> , positive if <code>s1 > s2</code> .
<code>strchr(s, c)</code> / <code>strrchr(s, c)</code>	Pointer to the first / last occurrence of character <code>c</code> in <code>s</code> , or <code>NULL</code> .
<code>strstr(s1, s2)</code>	Pointer to the first occurrence of substring <code>s2</code> inside <code>s1</code> , or <code>NULL</code> .
<code>strspn</code> , <code>strcspn</code> , <code>strpbrk</code> , <code>strtok</code>	Span character sets, find characters, and split (tokenise) a string using delimiters.

```

#include <stdio.h>
#include <string.h>

int main() {
    char a[20] = "Hello ";
    char b[] = "World";
    char c[20];

    printf("strlen(b) = %zu\n", strlen(b));

    strcpy(c, a);                /* c = "Hello " */
    strcat(c, b);                /* c = "Hello World" */
    printf("After strcpy + strcat: %s\n", c);

    if (strcmp("apple", "apple") == 0) printf("\"apple\" equals \"apple\"\n");
    if (strcmp("apple", "banana") < 0) printf("\"apple\" comes before \"banana\"\n");

    char *p = strchr(c, 'W');     /* pointer to 'W' */
    printf("'W' found at index %ld\n", (long)(p - c));

    char *q = strstr(c, "lo W");  /* pointer to the substring */
    printf("strstr gives: %s\n", q);
    return 0;
}

```

Output:

```

strlen(b) = 5
After strcpy + strcat: Hello World
"apple" equals "apple"
"apple" comes before "banana"
'W' found at index 6
strstr gives: lo World

```

****Splitting a string with `strtok()`:** the first call passes the string, later calls pass `NULL` to continue from where the last call stopped. `strtok` modifies the string, so it must be a modifiable array.

```

#include <stdio.h>
#include <string.h>

int main() {
    char data[] = "red,green,blue";
    char *token = strtok(data, ",");    /* first token */

    while (token != NULL) {
        printf("Token: %s\n", token);
        token = strtok(NULL, ",");     /* next tokens */
    }
    return 0;
}

```

Output:

```

Token: red
Token: green
Token: blue

```

CAUTION: `strcpy` and `strcat` do **not** check that the destination is large enough; too small a buffer causes a **buffer overflow**. Prefer bounded versions or make sure destinations are big enough. Note that `strncpy` does not add `'\0'` when the source is `n` characters or longer, so terminate the result manually.

4.4 Writing Your Own String Functions

Because strings are arrays ending in `'\0'`, most string operations are simple loops. This example combines **functions**, **arrays** and **strings**: length, reverse and palindrome check.

```
#include <stdio.h>

int myStrlen(char s[]) {                /* count characters until '\0' */
    int i = 0;
    while (s[i] != '\0')
        i++;
    return i;
}

void reverse(char s[]) {                /* reverse in place: modifies caller's array */
    int i = 0, j = myStrlen(s) - 1;
    while (i < j) {
        char temp = s[i];
        s[i] = s[j];
        s[j] = temp;
        i++;
        j--;
    }
}

int isPalindrome(char s[]) {
    int i = 0, j = myStrlen(s) - 1;
    while (i < j) {
        if (s[i] != s[j]) return 0;
        i++;
        j--;
    }
    return 1;
}

int main() {
    char word[] = "madam";
    char text[] = "programming";

    printf("Length of \"%s\" = %d\n", text, myStrlen(text));
    reverse(text);
    printf("Reversed: %s\n", text);
    printf("\"%s\" palindrome? %s\n", word, isPalindrome(word) ? "Yes" : "No");
    printf("\"%s\" palindrome? %s\n", text, isPalindrome(text) ? "Yes" : "No");
    return 0;
}
```

Output:

```
Length of "programming" = 11
Reversed: gnimmargorp
"madam" palindrome? Yes
"gnimmargorp" palindrome? No
```

4.5 Array of Strings

An array of strings is a **2D character array**: each row is one complete string.

```
#include <stdio.h>
#include <string.h>

int main() {
    char days[3][10] = {"Monday", "Tuesday", "Wednesday"};

    printf("Day 2 is: %s\n", days[1]);
    for (int i = 0; i < 3; i++)
        printf("days[%d] = %-10s length = %zu\n", i, days[i], strlen(days[i]));
    return 0;
}
```

Output:

```
Day 2 is: Tuesday
days[0] = Monday    length = 6
days[1] = Tuesday    length = 7
days[2] = Wednesday length = 9
```

In `char days[3][10]`, each row holds up to 9 characters plus the terminator. Every row is 10 bytes wide even for a short string, which **wastes memory** when strings differ in length. An **array of pointers** (Topic 5.5) avoids this.

```
#include <stdio.h>

int main() {
    char fixed[3][10] = {"Mon", "Tuesday", "Wednesday"}; /* 3 x 10 bytes */
    char *flex[3]      = {"Mon", "Tuesday", "Wednesday"}; /* 3 pointers */

    printf("2D char array size : %zu bytes\n", sizeof(fixed));
    printf("Pointer array size : %zu bytes (plus the literals themselves)\n", sizeof(flex));
    return 0;
}
```

Output:

```
2D char array size : 30 bytes
Pointer array size : 24 bytes (plus the literals themselves)
```

Topic 5: Pointers

A **pointer** is a variable that stores the **memory address** of another variable instead of a data value. Because a pointer gives direct access to a memory location, it enables efficient memory handling, dynamic memory allocation (Topic 9) and linked data structures.

5.1 Address-of and Dereference Operators

- **Address-of operator:** `&`, placed before a variable, gives the address where the variable is stored.
- **Dereference (indirection) operator:** `*`, placed before a pointer, reads or writes the value stored at the address the pointer holds.
- **Declaration:** a pointer is declared for one specific type, e.g. `int *ptr;` can validly hold only the address of an `int`.

Expression	Meaning	Value in the example below
<code>num</code>	The variable's value	10
<code>&num</code>	Address of <code>num</code>	1000 (example address)
<code>ptr</code>	Address stored in the pointer	1000
<code>*ptr</code>	Value at that address	10
<code>&ptr</code>	Address of the pointer variable itself	2000 (example address)

```
#include <stdio.h>

int main() {
    int num = 10;
    int *ptr = &num;           /* ptr holds the address of num */

    printf("Value of num      : %d\n", num);
    printf("Value via *ptr    : %d\n", *ptr);
    printf("ptr == &num ?    : %d\n", ptr == &num);

    *ptr = 20;                 /* changing num through the pointer */
    printf("num after *ptr = 20: %d\n", num);
    return 0;
}
```

Output:

```
Value of num      : 10
Value via *ptr    : 10
ptr == &num ?    : 1
num after *ptr = 20: 20
```

```
#include <stdio.h>

int main() {
    int num = 10;
    int *ptr = &num;
    printf("Address of num: %p\n", (void *)ptr);
    printf("Address of ptr: %p\n", (void *)&ptr);
    return 0;
}
```

Output:

```
Address of num: 0x7ffd5c3a1a4c
Address of ptr: 0x7ffd5c3a1a50      (addresses differ from run to run)
```

NULL and wild pointers

- A pointer with no valid target should be set to **NULL** (`int *p = NULL;`). Dereferencing **NULL** crashes immediately, which is easy to detect.
- An **uninitialised (wild) pointer** holds a random address; writing through it is undefined behaviour. Always initialise pointers.
- On a 64-bit system every pointer is 8 bytes, whatever type it points to.

5.2 Pointer to Array and Pointer Arithmetic

The name of an array, used alone, behaves as a **constant pointer to its first element**. So an array can be assigned to a pointer without `&`:

```
int arr[5] = {10, 20, 30};
int *p = arr;           /* same as: int *p = &arr[0]; */
```

When a value is added to a pointer, C **scales it by the size of the pointed-to type**. Adding 1 to an `int *` moves it forward by `sizeof(int)` bytes (typically 4), landing exactly on the next integer.

Operation	Meaning
<code>p++ / p--</code>	Move to the next / previous element.
<code>p + n, p - n</code>	Address of the element <code>n</code> positions ahead / behind. <code>p</code> itself is not changed.
<code>p2 - p1</code>	Number of elements between two pointers into the same array.
<code>*(p + i)</code>	Value of the <code>i</code> -th element (identical to <code>p[i]</code>).
<code>p1 < p2, p1 == p2</code>	Comparison of positions inside the same array.

```
#include <stdio.h>

int main() {
    int arr[4] = {10, 20, 30, 40};
    int *p = arr;                               /* points to arr[0] */

    for (int i = 0; i < 4; i++)
        printf("(p + %d) = %d\n", i, *(p + i));

    p++;                                         /* now points to arr[1] */
    printf("After p++, *p = %d\n", *p);

    p = p + 2;                                  /* now points to arr[3] */
    printf("*p = %d\n", *p);
    printf("Elements between arr and p : %d\n", p - arr);
    printf("Bytes between arr and p   : %d\n", (char *)p - (char *)arr);
    return 0;
}
```

Output:

```
*(p + 0) = 10
*(p + 1) = 20
*(p + 2) = 30
*(p + 3) = 40
After p++, *p = 20
*p = 40
Elements between arr and p : 3
Bytes between arr and p   : 12
```

Array-pointer equivalences

Array notation	Pointer notation	Meaning
<code>arr[i]</code>	<code>*(arr + i)</code>	value of the i-th element
<code>&arr[i]</code>	<code>arr + i</code>	address of the i-th element
<code>arr[0]</code>	<code>*arr</code>	first element
<code>p[i]</code>	<code>*(p + i)</code>	a pointer can also be indexed like an array

CAUTION: The array name is a **constant**: `arr++` is illegal, but `p++` is fine. Also, `sizeof(arr)` gives the whole array size (20 bytes for 5 ints) while `sizeof(p)` gives only the pointer size (8 bytes).

```
#include <stdio.h>

int main() {
    int arr[5] = {1, 2, 3, 4, 5};
    int *p = arr;
    printf("sizeof(arr) = %zu bytes\n", sizeof(arr));
    printf("sizeof(p)   = %zu bytes\n", sizeof(p));

    int sum = 0;
    for (p = arr; p < arr + 5; p++)          /* walk with the pointer itself */
        sum += *p;
    printf("Sum = %d\n", sum);
    return 0;
}
```

Output:

```
sizeof(arr) = 20 bytes
sizeof(p)   = 8 bytes
Sum = 15
```

Pointers and 2D arrays

Because a 2D array is stored in row-major order, `matrix[i]` is the i-th **row** (itself an array), and `matrix[i][j]` is the same as `*(*(matrix + i) + j)`.

```
#include <stdio.h>

int main() {
    int matrix[2][3] = { {1, 2, 3}, {4, 5, 6} };
    int (*rowPtr)[3] = matrix;          /* pointer to an array of 3 ints */

    printf("matrix[1][2]      = %d\n", matrix[1][2]);
    printf("*(*(matrix + 1) + 2) = %d\n", *(*(matrix + 1) + 2));
    printf("rowPtr[0][1]      = %d\n", rowPtr[0][1]);

    int *flat = &matrix[0][0];          /* walk all 6 elements in memory order */
    for (int i = 0; i < 6; i++)
        printf("%d ", flat[i]);
    printf("\n");
    return 0;
}
```

Output:

```
matrix[1][2]      = 6
*(*(matrix + 1) + 2) = 6
rowPtr[0][1]      = 2
1 2 3 4 5 6
```

5.3 Pointers and Variable-Length Data: Array of Pointers

An **array of pointers** is an array whose every element is itself a pointer. It is especially useful for a collection of **variable-length strings**: each pointer refers to a string of exactly the length it needs, unlike a fixed-width 2D char array.

```
#include <stdio.h>
#include <string.h>

int main() {
    char *names[] = {"Alice", "Bob", "Charlie"};    /* array of pointers to strings */
    int count = sizeof(names) / sizeof(names[0]);

    for (int i = 0; i < count; i++)
        printf("%d. %s (%zu letters)\n", i + 1, names[i], strlen(names[i]));
    return 0;
}
```

Output:

```
1. Alice (5 letters)
2. Bob (3 letters)
3. Charlie (7 letters)
```

An array of pointers can also point to ordinary variables:

```
#include <stdio.h>

int main() {
    int a = 10, b = 20, c = 30;
    int *ptrs[3] = {&a, &b, &c};

    *ptrs[1] = 99;    /* changes b through the pointer */
    for (int i = 0; i < 3; i++)
        printf("*ptrs[%d] = %d\n", i, *ptrs[i]);
    printf("b is now %d\n", b);
    return 0;
}
```

Output:

```
*ptrs[0] = 10
*ptrs[1] = 99
*ptrs[2] = 30
b is now 99
```

5.4 Pointer to Pointer (Double Pointer)

A **pointer to a pointer** stores the address of another pointer. Each extra level of indirection adds one asterisk: `int **pp;`. It is used when a function must change a **pointer itself** (Topic 9.5), and for dynamically allocated 2D arrays and arrays of strings.

```
#include <stdio.h>

int main() {
    int var = 50;
    int *ptr1 = &var;          /* pointer to var */
    int **ptr2 = &ptr1;        /* pointer to pointer ptr1 */

    printf("var      = %d\n", var);
    printf("*ptr1    = %d\n", *ptr1);      /* one dereference -> var */
    printf("**ptr2   = %d\n", **ptr2);     /* two dereferences -> var */

    **ptr2 = 75;                /* modify var through ptr2 */
    printf("var now = %d\n", var);
    return 0;
}
```

Output:

```
var      = 50
*ptr1    = 50
**ptr2   = 50
var now = 75
```

`*ptr2` gives back `ptr1` (the address of `var`); `**ptr2` follows that address on to the value of `var` itself.

TIP: Pointers to **structures** (`->` operator), pointers **inside** structures and self-referential structures are covered with structures in Topic 10.5. The **dangling pointer** problem is covered with dynamic memory in Topic 9.6.

Topic 6: Parameter Passing and Returning Values

How data moves between a function and its caller decides whether changes made inside a function are visible outside it. This topic brings together functions, pointers and arrays.

6.1 What Can Be Passed to a Function

Passed argument	How it is passed	Can the function change the caller's data?
Basic types (<code>int</code> , <code>float</code> , <code>char</code> , <code>double</code>)	By value (a copy)	No
Pointer (<code>&x</code>)	Address is copied; the function reaches the original through it	Yes
Array name	Decays to a pointer to its first element	Yes (elements are shared)
Structure	By value (full copy) or by pointer	Only when passed by pointer

6.2 Call by Value and Call by Reference

Call by value

The function receives an **independent copy** of the argument, stored in its own memory location (the local parameter). Changes made to the parameter never affect the original variable; the copy is discarded when the function returns.

```
#include <stdio.h>

void addTen(int x) {
    x = x + 10;           /* changes only the local copy */
    printf("Inside function : x = %d\n", x);
}

int main() {
    int num = 5;
    addTen(num);
    printf("Back in main    : num = %d\n", num);    /* unchanged */
    return 0;
}
```

Output:

```
Inside function : x = 15
Back in main    : num = 5
```

Call by reference (passing addresses)

The function is given the **address** of the original variable (obtained with **&**). By dereferencing the pointer (***x**) it modifies the caller's variable directly, and the change remains after the function returns.

```
#include <stdio.h>

void addTen(int *x) {
    *x = *x + 10;           /* dereference: changes the original */
}

int main() {
    int num = 5;
    addTen(&num);          /* pass the address */
    printf("num = %d\n", num); /* 15 */
    return 0;
}
```

Output:

```
num = 15
```

The classic example: swapping two numbers

```
#include <stdio.h>

void swapByValue(int a, int b) {           /* swaps only the copies */
    int temp = a;
    a = b;
    b = temp;
}

void swapByReference(int *a, int *b) {     /* swaps the originals */
    int temp = *a;
    *a = *b;
    *b = temp;
}

int main() {
    int x = 10, y = 20;

    swapByValue(x, y);
    printf("After swapByValue    : x = %d, y = %d\n", x, y);

    swapByReference(&x, &y);
    printf("After swapByReference: x = %d, y = %d\n", x, y);
    return 0;
}
```

Output:

```
After swapByValue    : x = 10, y = 20
After swapByReference: x = 20, y = 10
```

Aspect	Call by Value	Call by Reference
What is passed	A copy of the value	The address (pointer) of the original variable
Effect on original	None	Original is modified directly
Memory	Extra memory for the copy	Only the address is copied
Syntax cue	<code>void f(int x)</code> called as <code>f(a)</code>	<code>void f(int *x)</code> called as <code>f(&a)</code>
Typical use	Function only reads the value	Function must modify the caller's variable, or return several values

NOTE: Strictly speaking, C always passes arguments **by value**. "Call by reference" is achieved by passing the **address by value** and dereferencing it. This is the standard way the term is used in C notes and exams.

6.3 Passing Arrays to Functions

An array name decays to a pointer to its first element, so the function works on the **original elements**, and changes are visible in the caller. No copy of the array is made. The **size is not passed automatically**: inside the function `sizeof(arr)` gives only the pointer size, so the length must be passed as a separate parameter.

```
#include <stdio.h>

int sumArray(int arr[], int n) {           /* int arr[] is the same as int *arr */
    int sum = 0;
    for (int i = 0; i < n; i++)
        sum += arr[i];
    return sum;
}

void doubleAll(int arr[], int n) {         /* modifies the caller's array */
    for (int i = 0; i < n; i++)
        arr[i] *= 2;
}

int main() {
    int data[4] = {1, 2, 3, 4};

    printf("Sum before : %d\n", sumArray(data, 4));
    doubleAll(data, 4);
    printf("Sum after : %d\n", sumArray(data, 4));
    printf("data[] = %d %d %d %d\n", data[0], data[1], data[2], data[3]);
    return 0;
}
```

Output:

```
Sum before : 10
Sum after : 20
data[] = 2 4 6 8
```

Passing a 2D array: every dimension except the first must be given, so the compiler can compute element addresses.

```
#include <stdio.h>

void printMatrix(int m[][3], int rows) {   /* column count (3) is required */
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < 3; j++)
            printf("%3d", m[i][j]);
        printf("\n");
    }
}

int main() {
    int matrix[2][3] = { {1, 2, 3}, {4, 5, 6} };
    printMatrix(matrix, 2);
    return 0;
}
```

Output:

```
1 2 3
4 5 6
```

6.4 Returning Values

- A function returns **one value** with **return**; its type must match the declared return type.
- To hand back **several results**, pass pointers as parameters and write the results through them.
- A function may also **return a pointer**, but never the address of one of its own local variables (Topic 9.6).

Returning multiple results through pointers:

```
#include <stdio.h>

void minMax(int a, int b, int *minOut, int *maxOut) {
    if (a < b) { *minOut = a; *maxOut = b; }
    else      { *minOut = b; *maxOut = a; }
}

int main() {
    int small, large;
    minMax(15, 7, &small, &large);
    printf("Min: %d, Max: %d\n", small, large);
    return 0;
}
```

Output:

```
Min: 7, Max: 15
```

Returning a pointer to data that outlives the function (an element of the caller's array):

```
#include <stdio.h>

int *findMax(int arr[], int n) {
    int *maxPtr = &arr[0];
    for (int i = 1; i < n; i++)
        if (arr[i] > *maxPtr)
            maxPtr = &arr[i];
    return maxPtr;
}

int main() {
    int marks[5] = {55, 72, 88, 61, 79};
    int *best = findMax(marks, 5);
    printf("Highest = %d at index %d\n", *best, best - marks);
    *best = 100;
    printf("marks[2] = %d\n", marks[2]);
    return 0;
}
```

Output:

```
Highest = 88 at index 2
marks[2] = 100
```

TIP: Passing and returning **structures** by value and by pointer is shown in Topic 10.6.

Topic 7: Recursion

Recursion occurs when a function **calls itself**, either directly or indirectly (function A calls B, which calls A), to solve a smaller instance of the same problem.

7.1 The Two Parts of a Recursive Function

- **Base condition (base case):** the situation in which the function stops calling itself and returns a direct answer. **Every recursive function needs at least one.**
- **Recursive case:** the function calls itself with a *smaller* or simpler input that moves toward the base case.

CAUTION: Without a base condition (or if the calls never reach it) the function keeps calling itself, filling the call stack until the program crashes with a **stack overflow**.

7.2 Factorial

$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$, and in general $n! = n \times (n-1)!$, with $0! = 1! = 1$.

```
#include <stdio.h>

int factorial(int n) {
    if (n == 0 || n == 1)          /* base condition */
        return 1;
    return n * factorial(n - 1); /* recursive call */
}

int main() {
    printf("Factorial of 5 is: %d\n", factorial(5));
    return 0;
}
```

Output:

Factorial of 5 is: 120

How the calls unfold (call-stack trace):

```
factorial(5)
= 5 * factorial(4)
= 5 * (4 * factorial(3))
= 5 * (4 * (3 * factorial(2)))
= 5 * (4 * (3 * (2 * factorial(1))))
= 5 * (4 * (3 * (2 * 1)))      <- base case returns 1
= 120
```

Each call waits, paused on the call stack, for the next call to return. Once **factorial(1)** returns 1, the chain unwinds and each level multiplies its result.

7.3 More Recursion Examples

```
#include <stdio.h>

int fib(int n) {
    /* Fibonacci: 0 1 1 2 3 5 8 ... */
    if (n == 0) return 0;          /* base cases */
    if (n == 1) return 1;
    return fib(n - 1) + fib(n - 2);
}

int sumDigits(int n) {
    if (n == 0) return 0;          /* base case */
    return (n % 10) + sumDigits(n / 10);
}

int gcd(int a, int b) {
    /* Euclid's algorithm */
    if (b == 0) return a;          /* base case */
    return gcd(b, a % b);
}

int power(int base, int exp) {
    if (exp == 0) return 1;        /* base case */
    return base * power(base, exp - 1);
}

int main() {
    printf("Fibonacci series: ");
    for (int i = 0; i < 8; i++)
        printf("%d ", fib(i));
    printf("\nsumDigits(4729) = %d\n", sumDigits(4729));
    printf("gcd(48, 18)      = %d\n", gcd(48, 18));
    printf("power(2, 10)     = %d\n", power(2, 10));
    return 0;
}
```

Output:

```
Fibonacci series: 0 1 1 2 3 5 8 13
sumDigits(4729) = 22
gcd(48, 18)     = 6
power(2, 10)    = 1024
```

Recursion with arrays and strings: the problem shrinks by one element each call.

```
#include <stdio.h>

int sumArray(int arr[], int n) {
    /* sum of first n elements */
    if (n == 0) return 0;        /* base case: empty array */
    return arr[n - 1] + sumArray(arr, n - 1);
}

void printReverse(char *s) {
    /* prints a string backwards */
    if (*s == '\0') return;      /* base case: end of string */
    printReverse(s + 1);          /* first handle the rest ... */
    putchar(*s);                  /* ... then print this char */
}

int main() {
    int a[5] = {3, 6, 9, 12, 15};
    printf("Sum = %d\n", sumArray(a, 5));
    printf("Reversed: ");
    printReverse("recursion");
    printf("\n");
    return 0;
}
```

Output:

```
Sum = 45
Reversed: noisrucer
```

Indirect recursion

```
#include <stdio.h>

int isOdd(int n);           /* prototype needed: used before defined */

int isEven(int n) {
    if (n == 0) return 1;
    return isOdd(n - 1);    /* isEven calls isOdd ... */
}

int isOdd(int n) {
    if (n == 0) return 0;
    return isEven(n - 1);   /* ... which calls isEven */
}

int main() {
    printf("isEven(6) = %d, isEven(7) = %d\n", isEven(6), isEven(7));
    return 0;
}
```

Output:

```
isEven(6) = 1, isEven(7) = 0
```

7.4 Recursion vs Iteration

Aspect	Recursion	Iteration (loops)
Code length	Often shorter and more natural for tree-like or mathematically defined problems	Can be longer for such problems
Memory	More: every call adds a frame to the call stack	Less: a fixed amount of memory
Speed	Function-call overhead on every step	Usually faster
Termination	Needs a base condition	Needs a loop condition that eventually becomes false
Risk	Stack overflow if too deep or no base case	Infinite loop if the update is missing

Any recursive algorithm can be rewritten with iteration, sometimes at the cost of a less intuitive program.

Topic 8: Storage Classes

A **storage class** defines three properties of a variable: its **scope** (where it can be accessed), its **lifetime** (how long it stays in memory) and its **default initial value**.

Storage class	Keyword	Scope	Lifetime	Default value
Automatic	<code>auto</code>	Local (block / function)	Until the block or function exits	Garbage (indeterminate)
Register	<code>register</code>	Local	Until the block exits; requested to be kept in a CPU register	Garbage
Static	<code>static</code>	Local (but keeps its value between calls)	Entire program run	Zero
External	<code>extern</code>	Global (shareable across source files)	Entire program run	Zero

8.1 auto

The **default** class for every variable declared inside a function or block. It is created when the block is entered and destroyed when the block ends, so it is **not** visible outside it.

```
#include <stdio.h>

int main() {
    int a = 10;                /* same as: auto int a = 10; */
    {
        int b = 20;           /* exists only inside this block */
        printf("Inside block : a = %d, b = %d\n", a, b);
    }
    printf("Outside block: a = %d\n", a);
    /* printf("%d", b); would be a compile error: b is out of scope here */
    return 0;
}
```

Output:

```
Inside block : a = 10, b = 20
Outside block: a = 10
```

8.2 register

A **hint** to the compiler that the variable is used very often and should be kept in a CPU register for speed. Modern compilers optimise this automatically and may ignore the hint. The address of a **register** variable cannot be taken with `&`.

```
#include <stdio.h>

int main() {
    register int i;            /* frequently used loop counter */
    long total = 0;

    for (i = 1; i <= 100; i++)
        total += i;
    printf("Sum of 1..100 = %ld\n", total);
    /* &i would be an error: a register variable has no guaranteed address */
    return 0;
}
```

Output:

```
Sum of 1..100 = 5050
```

8.3 static

A **static local variable** is initialised **only once**, on the first call, and then **retains its value between calls**. An ordinary (auto) variable is recreated fresh on every call.

```
#include <stdio.h>

void demo() {
    int a = 0;           /* auto: re-created and reset on every call */
    static int s = 0;     /* static: initialised once, value persists */
    a++;
    s++;
    printf("auto a = %d, static s = %d\n", a, s);
}

int main() {
    demo();
    demo();
    demo();
    return 0;
}
```

Output:

```
auto a = 1, static s = 1
auto a = 1, static s = 2
auto a = 1, static s = 3
```

A **static** variable declared **outside** any function (or a **static** function) is visible only inside the file that defines it.

8.4 extern

extern declares a variable that is **defined elsewhere**: later in the same file, or in a different source file. It lets one global variable be shared by several files.

```
#include <stdio.h>

int globalCount = 100;           /* global variable: defined once */

void show() {
    extern int globalCount;       /* declaration only: no new variable */
    globalCount++;
    printf("show(): globalCount = %d\n", globalCount);
}

int main() {
    show();
    printf("main(): globalCount = %d\n", globalCount);
    return 0;
}
```

Output:

```
show(): globalCount = 101
main(): globalCount = 101
```

Sharing a variable between two source files:**Compile together with: gcc file1.c file2.c (output: counter = 2)**

```

/* ----- file1.c ----- */
#include <stdio.h>
int counter = 0;           /* the one and only definition */
void increment(void);      /* function defined in file2.c */

int main() {
    increment();
    increment();
    printf("counter = %d\n", counter);
    return 0;
}

/* ----- file2.c ----- */
extern int counter;        /* refers to the variable defined in file1.c */
void increment(void) {
    counter++;
}

```

8.5 Summary of Storage Classes

Feature	auto	register	static	extern
Stored in	Stack	CPU register (if possible)	Data segment	Data segment
Default value	Garbage	Garbage	0	0
Scope	Block	Block	Block (or file if global)	Whole program
Lifetime	Block	Block	Whole program	Whole program
Keeps value between calls?	No	No	Yes	Yes

Topic 9: Dynamic Memory Allocation

Ordinary variables and arrays get their memory automatically, and the size of an array is fixed at compile time. **Dynamic memory allocation** reserves and releases memory **explicitly at run time**. It is essential when the amount of memory needed is not known until the program runs (for example, an array whose size the user types in). Dynamic memory is accessed **through pointers**, which is why this topic sits between Pointers and Structures.

9.1 Stack vs Heap

Feature	Stack (automatic memory)	Heap (dynamic memory)
Used for	Local variables, function parameters, call frames	Blocks obtained with <code>malloc</code> , <code>calloc</code> , <code>realloc</code>
Allocated by	The compiler, automatically	The programmer, at run time
Released	Automatically when the function / block exits	Only when the programmer calls <code>free()</code>
Size	Fixed at compile time (limited)	Decided at run time (limited only by available memory)
Accessed via	Variable names	Pointers

9.2 Memory Allocation Functions (stdlib.h)

Function	Description
<code>malloc(size)</code>	Allocates one block of <code>size</code> bytes. The memory is not initialised (contains garbage). Returns <code>NULL</code> on failure.
<code>calloc(n, size)</code>	Allocates memory for n blocks , each of <code>size</code> bytes, and sets every byte to zero .
<code>realloc(ptr, new_size)</code>	Resizes a block obtained earlier; existing contents are preserved up to the smaller of the old and new sizes.
<code>free(ptr)</code>	Releases dynamically allocated memory back to the system. Forgetting it causes a memory leak .

	<code>malloc</code>	<code>calloc</code>
Arguments	One: total bytes <code>malloc(n * sizeof(int))</code>	Two: count and size <code>calloc(n, sizeof(int))</code>
Initial contents	Garbage	All zeros
Speed	Slightly faster (no zeroing)	Slightly slower (zeroing)

TIP: Rules: (1) always check the result against `NULL`; (2) call `free()` exactly once for every successful allocation; (3) never use memory after freeing it; (4) `sizeof` should always be used to compute sizes, never hard-coded byte counts.

9.3 Dynamic Array with malloc

The size is read from the user at run time, then exactly that much memory is requested.

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n;
    scanf("%d", &n);                                /* size known only at run time */

    int *arr = (int *)malloc(n * sizeof(int));        /* block for n integers          */
    if (arr == NULL) {                                /* always check for failure      */
        printf("Memory allocation failed!\n");
        return 1;
    }

    int sum = 0;
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);                        /* use it like a normal array    */
        sum += arr[i];
    }
    printf("Sum of %d numbers = %d\n", n, sum);

    free(arr);                                         /* release the memory            */
    arr = NULL;                                       /* avoid a dangling pointer      */
    return 0;
}
```

Sample input:

```
5
10 20 30 40 50
```

Output:

```
Sum of 5 numbers = 150
```

9.4 calloc and realloc

calloc gives zero-filled memory. **realloc** changes the size of an existing block; store its result in a **temporary** pointer first, because on failure it returns **NULL** and the original block is still valid (and still needs **free**).

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *arr = (int *)calloc(3, sizeof(int));           /* 3 ints, all set to 0 */
    if (arr == NULL) return 1;

    printf("Fresh calloc memory: %d %d %d\n", arr[0], arr[1], arr[2]);
    for (int i = 0; i < 3; i++)
        arr[i] = (i + 1) * 10;

    int *tmp = (int *)realloc(arr, 6 * sizeof(int));    /* grow from 3 to 6 ints */
    if (tmp == NULL) {                                  /* failure: keep old block */
        free(arr);
        return 1;
    }
    arr = tmp;
    for (int i = 3; i < 6; i++)                          /* fill the new elements */
        arr[i] = (i + 1) * 10;

    printf("After realloc      : ");
    for (int i = 0; i < 6; i++)
        printf("%d ", arr[i]);
    printf("\n");

    free(arr);
    return 0;
}
```

Output:

```
Fresh calloc memory: 0 0 0
After realloc      : 10 20 30 40 50 60
```

9.5 Dynamic Strings and Dynamic 2D Arrays

Dynamic string: allocate for the **final** size, which is the length of the text plus one extra byte for **'\0'**. **strcpy** and **strcat** do not check space, so too small a block causes a buffer overflow.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main() {
    char *src = "Dynamic memory";
    char *extra = " in C";
    char *text = (char *)malloc(strlen(src) + strlen(extra) + 1); /* final size */
    if (text == NULL) return 1;

    strcpy(text, src);
    strcat(text, extra);
    printf("%s (length %zu)\n", text, strlen(text));
    free(text);
    return 0;
}
```

Output:

```
Dynamic memory in C (length 19)
```

Dynamic 2D array using a pointer to pointer (Topic 5.4): first an array of row pointers, then one block per row.

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int rows = 3, cols = 4;
    int **m = (int **)malloc(rows * sizeof(int *));    /* array of row pointers */
    for (int i = 0; i < rows; i++)
        m[i] = (int *)malloc(cols * sizeof(int));    /* each row has cols ints */

    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            m[i][j] = (i + 1) * (j + 1);    /* multiplication table */

    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++)
            printf("%3d", m[i][j]);
        printf("\n");
    }

    for (int i = 0; i < rows; i++)    /* free rows first ... */
        free(m[i]);
    free(m);    /* ... then the pointer array */
    return 0;
}
```

Output:

```
1  2  3  4
2  4  6  8
3  6  9 12
```

Why a pointer to pointer? To let a function allocate memory for the *caller's* pointer, the function must receive the **address of that pointer**.

```
#include <stdio.h>
#include <stdlib.h>

void allocate(int **p, int n) {    /* p is the address of the caller's pointer */
    *p = (int *)malloc(n * sizeof(int));    /* changes the caller's pointer itself */
    for (int i = 0; i < n; i++)
        (*p)[i] = i * i;
}

int main() {
    int *data = NULL;
    allocate(&data, 5);    /* pass the address of the pointer */

    for (int i = 0; i < 5; i++)
        printf("%d ", data[i]);
    printf("\n");
    free(data);
    return 0;
}
```

Output:

```
0 1 4 9 16
```

9.6 Memory Leak and the Dangling Pointer Problem

A **memory leak** occurs when allocated memory is never freed: the program keeps using more and more memory. A **dangling pointer** is a pointer that still holds the address of memory that has been **freed** or has gone **out of scope**. Reading or writing through it is **undefined behaviour**: the program may crash, corrupt other data, or seem to work by coincidence and fail later.

Common causes of dangling pointers

- Using a pointer after `free()` has released its memory.
- Returning the address of a **local variable** from a function: its stack memory is invalid once the function returns.
- Two pointers refer to the same block; it is freed through one and the other is still used.

Examples of bugs (do not run):

```
/* CAUSE 1: use after free */
int *p = (int *)malloc(sizeof(int));
*p = 10;
free(p);
*p = 20;                /* BUG: p is dangling */

/* CAUSE 2: returning the address of a local variable */
int *bad(void) {
    int x = 10;
    return &x;           /* BUG: x is destroyed when bad() returns */
}

/* CAUSE 3: two pointers, one block */
int *a = (int *)malloc(sizeof(int));
int *b = a;
free(a);
printf("%d", *b);        /* BUG: b is dangling too */
```

Fixes

- Immediately after `free(p)`, set `p = NULL`. Using a `NULL` pointer crashes at once and is easy to detect, whereas using a dangling pointer silently corrupts memory. Calling `free(NULL)` is harmless, which also prevents a double free.
- Return dynamically allocated memory (the caller frees it), a `static` variable, or a pointer into the caller's data, never the address of a local variable.

```
#include <stdio.h>
#include <stdlib.h>

int *makeNumber(void) {
    int *p = (int *)malloc(sizeof(int)); /* heap memory outlives the function */
    if (p != NULL) *p = 10;
    return p;                             /* SAFE: caller now owns this memory */
}

int main() {
    int *ptr = makeNumber();
    printf("Value = %d\n", *ptr);

    free(ptr);
    ptr = NULL;                          /* no longer dangling */

    if (ptr == NULL)
        printf("ptr is NULL, so it cannot be misused by accident\n");
    return 0;
}
```

Output:

```
Value = 10
ptr is NULL, so it cannot be misused by accident
```

Topic 10: Structures

Primary types such as **int**, **float** and **char** hold a single value, and arrays hold many values of the **same** type. Real-world records combine **different** kinds of data: a student has an integer roll number, a name (text) and floating-point marks. A **structure** (**struct**) groups items of different types under **one name**, so that a record can be handled as one unit.

10.1 Declaring a Structure

First the **blueprint** is defined. This tells the compiler which members the structure has, but it does **not** create a variable or reserve memory.

```
struct Book {
    int    book_id;
    char   title[50];
    float  price;
};          /* the semicolon after the closing brace is required */
```

Variables of that type are then declared:

```
struct Book b1, b2;          /* after the blueprint */

struct Student {            /* blueprint and variables in one statement */
    int roll_no;
    float marks;
} s1, s2;
```

10.2 Initialising and Accessing Members

A structure variable can be initialised when it is declared (values in member order, inside braces), or its members can be assigned one by one with the **dot operator** (**.**). C99 also allows **designated initialisers** (**.member = value**), in any order.

```
#include <stdio.h>
#include <string.h>

struct Book {
    int    book_id;
    char   title[50];
    float  price;
};

int main() {
    struct Book b1 = {101, "C Programming", 299.50};    /* in member order */

    struct Book b2;                                     /* assign one by one */
    b2.book_id = 102;
    strcpy(b2.title, "Data Structures");                /* strings need strcpy */
    b2.price = 350.00;

    struct Book b3 = { .price = 425.75, .title = "Algorithms", .book_id = 103 }; /* C99 */

    printf("%d %-16s Rs.%.2f\n", b1.book_id, b1.title, b1.price);
    printf("%d %-16s Rs.%.2f\n", b2.book_id, b2.title, b2.price);
    printf("%d %-16s Rs.%.2f\n", b3.book_id, b3.title, b3.price);
    return 0;
}
```

Output:

```
101 C Programming    Rs.299.50
102 Data Structures  Rs.350.00
103 Algorithms       Rs.425.75
```

NOTE: **Why `strcpy()`?** `title` is a `char` array, and arrays cannot be assigned with `=` after declaration (only initialised at the moment of declaration). `strcpy()` from `<string.h>` copies the text in.

Reading a structure from the keyboard: each member is read separately. `%49[^\n]` reads a line of text with spaces, at most 49 characters.

```
#include <stdio.h>

struct Book {
    int    book_id;
    char   title[50];
    float  price;
};

int main() {
    struct Book b;
    scanf("%d", &b.book_id);
    scanf(" %49[^\n]", b.title);    /* no & for arrays; leading space skips newline */
    scanf("%f", &b.price);

    printf("ID: %d | Title: %s | Price: Rs.%.2f\n", b.book_id, b.title, b.price);
    return 0;
}
```

Sample input:

```
103
Programming in C
450.75
```

Output:

```
ID: 103 | Title: Programming in C | Price: Rs.450.75
```

10.3 Size, Copying and Comparing

- **Assignment** between two variables of the same structure type copies **every member** (even arrays inside the structure): `b2 = b1;`
- Structures **cannot be compared** with `==`; compare member by member.
- The size of a structure is at least the sum of its members' sizes. The compiler may add **padding** bytes so that members are aligned in memory.

```
#include <stdio.h>
#include <string.h>

struct Student { int roll; char name[20]; float marks; };
struct Padded { char c; int i; };          /* 1 + 4 = 5 bytes of data */

int main() {
    struct Student a = {1, "Riya", 85.5};
    struct Student b;
    b = a;                                /* copies all members, including name[] */
    strcpy(b.name, "Amit");               /* changing the copy leaves a unchanged */
    printf("a: %d %s   b: %d %s\n", a.roll, a.name, b.roll, b.name);

    if (a.roll == b.roll && strcmp(a.name, b.name) == 0 && a.marks == b.marks)
        printf("Same student\n");
    else
        printf("Different students (compared member by member)\n");

    printf("sizeof(struct Student) = %zu\n", sizeof(struct Student));
    printf("sizeof(struct Padded) = %zu (5 data bytes + padding)\n",
           sizeof(struct Padded));
    return 0;
}
```

Output:

```
a: 1 Riya   b: 1 Amit
Different students (compared member by member)
sizeof(struct Student) = 28
sizeof(struct Padded) = 8 (5 data bytes + padding)
```

10.4 Array of Structures

When many records share the same structure type (for example all 60 students of a class), an **array of structures** is used. Each element such as `cls[0]` is a complete structure; its members are reached with indexing plus the dot operator: `cls[0].roll_no`.

```
#include <stdio.h>

struct Student {
    int    roll_no;
    char   name[20];
    float  marks;
};

int main() {
    struct Student cls[3] = { {1, "Riya", 85.5}, {2, "Amit", 90.0}, {3, "Neha", 78.0} };
    float total = 0;
    int topper = 0;

    printf("Roll  Name    Marks\n");
    for (int i = 0; i < 3; i++) {
        printf("%-5d %-8s %5.1f\n", cls[i].roll_no, cls[i].name, cls[i].marks);
        total += cls[i].marks;
        if (cls[i].marks > cls[topper].marks)
            topper = i;
    }
    printf("Average = %.2f\n", total / 3);
    printf("Topper   = %s (%.1f)\n", cls[topper].name, cls[topper].marks);
    return 0;
}
```

Output:

```
Roll  Name    Marks
1     Riya     85.5
2     Amit     90.0
3     Neha     78.0
Average = 84.50
Topper   = Amit (90.0)
```

10.5 Pointers and Structures

(a) Pointer to a structure and the arrow operator

Members of a structure accessed **through a pointer** use the **arrow operator ->**. It combines dereferencing and member access: `ptr->x` is a shorthand for `(*ptr).x`.

```
#include <stdio.h>

struct Point { int x; int y; };

int main() {
    struct Point p1 = {10, 20};
    struct Point *ptr = &p1;

    printf("Using arrow      : x = %d, y = %d\n", ptr->x, ptr->y);
    printf("Using (*ptr).x   : x = %d, y = %d\n", (*ptr).x, (*ptr).y);

    ptr->x = 99;                      /* modifies p1 itself */
    printf("p1.x is now %d\n", p1.x);
    return 0;
}
```

Output:

```
Using arrow      : x = 10, y = 20
Using (*ptr).x   : x = 10, y = 20
p1.x is now 99
```

(b) Pointer within a structure

A structure may contain a **pointer member**. This is common for variable-length data such as strings, and for linking structures together.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Employee {
    int empId;
    char *name;           /* pointer within a structure */
};

int main() {
    struct Employee e1 = {101, "Kiran"};           /* points to a string literal */

    struct Employee e2;
    e2.empId = 102;
    e2.name = (char *)malloc(20);                  /* points to heap memory */
    strcpy(e2.name, "Meera Patel");

    printf("%d %s\n", e1.empId, e1.name);
    printf("%d %s\n", e2.empId, e2.name);

    free(e2.name);                                  /* free members first, if allocated */
    return 0;
}
```

Output:

```
101 Kiran
102 Meera Patel
```

(c) Walking through an array of structures with a pointer

```
#include <stdio.h>

struct Student { int roll_no; float marks; };

int main() {
    struct Student cls[3] = { {1, 85.5}, {2, 90.0}, {3, 78.0} };
    struct Student *p = cls;                /* points to cls[0] */

    for (int i = 0; i < 3; i++, p++)        /* p++ moves by one whole struct */
        printf("Roll %d -> %.1f\n", p->roll_no, p->marks);
    return 0;
}
```

Output:

```
Roll 1 -> 85.5
Roll 2 -> 90.0
Roll 3 -> 78.0
```

(d) Self-referential structure: a linked list node

A structure can contain a pointer to **its own type**. This is the building block of **linked lists**, trees and graphs, where nodes are joined by pointers instead of sitting in one contiguous block.

```
#include <stdio.h>

struct Node {
    int data;
    struct Node *next;        /* pointer to the next node (same type) */
};

int main() {
    struct Node n3 = {30, NULL}; /* last node: next is NULL */
    struct Node n2 = {20, &n3};
    struct Node n1 = {10, &n2};

    struct Node *cur = &n1;      /* start at the head */
    while (cur != NULL) {
        printf("%d -> ", cur->data);
        cur = cur->next;         /* follow the link */
    }
    printf("NULL\n");
    return 0;
}
```

Output:

```
10 -> 20 -> 30 -> NULL
```

10.6 Structures and Functions

The same value / reference rules from Topic 6 apply:

- **By value:** the whole structure is copied into the parameter; the original cannot be changed. Simple and safe for read-only work.
- **By reference (pointer):** only the address is passed. This avoids copying a large structure and lets the function modify the original via `->`.
- A function can also **return** a structure.

```
#include <stdio.h>

struct Rectangle { int length; int width; };

void printArea(struct Rectangle r) {           /* BY VALUE: read only */
    printf("Area is: %d\n", r.length * r.width);
}

void scaleRectangle(struct Rectangle *r, int factor) { /* BY REFERENCE */
    r->length *= factor;                          /* modifies original */
    r->width  *= factor;
}

int main() {
    struct Rectangle box = {5, 10};
    printArea(box);
    scaleRectangle(&box, 2);                    /* pass the address */
    printArea(box);
    return 0;
}
```

Output:

```
Area is: 50
Area is: 200
```

Returning a structure, and passing an array of structures:

```
#include <stdio.h>

struct Point { int x; int y; };
struct Student { char name[10]; float marks; };

struct Point addPoints(struct Point a, struct Point b) { /* returns a structure */
    struct Point r;
    r.x = a.x + b.x;
    r.y = a.y + b.y;
    return r;
}

float average(struct Student s[], int n) { /* array of structs: shared, not copied */
    float total = 0;
    for (int i = 0; i < n; i++)
        total += s[i].marks;
    return total / n;
}

int main() {
    struct Point p = addPoints((struct Point){1, 2}, (struct Point){10, 20});
    printf("Sum of points = (%d, %d)\n", p.x, p.y);

    struct Student cls[3] = { {"Riya", 80}, {"Amit", 90}, {"Neha", 70} };
    printf("Class average = %.1f\n", average(cls, 3));
    return 0;
}
```

Output:

```
Sum of points = (11, 22)
Class average = 80.0
```

10.7 Nested Structures

A structure can contain **another structure** as a member. For example, an Employee record naturally contains a Date structure for the joining date, instead of three loose integers. The inner structure must be defined **before** the outer one.

```
#include <stdio.h>

struct Date {                                /* inner structure */
    int day;
    int month;
    int year;
};

struct Employee {                            /* outer structure */
    int id;
    char name[20];
    struct Date join_date;                  /* nested structure */
};

int main() {
    struct Employee e1 = {101, "John", {15, 8, 2023}}; /* nested braces */

    printf("%s joined on %d/%d/%d\n", e1.name,
           e1.join_date.day, e1.join_date.month, e1.join_date.year); /* chained dots */

    struct Employee *ePtr = &e1;
    ePtr->join_date.year = 2024;             /* arrow, then dot */
    printf("Updated year via pointer: %d\n", ePtr->join_date.year);
    return 0;
}
```

Output:

```
John joined on 15/8/2023
Updated year via pointer: 2024
```

10.8 Dynamic Memory Allocation for Structures

A structure (or an **array of structures**) can be allocated on the heap when the number of records is not known in advance. `sizeof(struct ...)` tells `malloc` exactly how many bytes are needed.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Employee {
    int id;
    char name[20];
};

int main() {
    /* One structure */
    struct Employee *empPtr = (struct Employee *)malloc(sizeof(struct Employee));
    if (empPtr == NULL) {
        printf("Memory allocation failed!\n");
        return 1;
    }
    empPtr->id = 1;
    strcpy(empPtr->name, "Alice");
    printf("Employee ID: %d, Name: %s\n", empPtr->id, empPtr->name);
    free(empPtr);

    /* An array of n structures */
    int n = 3;
    struct Employee *team = (struct Employee *)calloc(n, sizeof(struct Employee));
    if (team == NULL) return 1;
    for (int i = 0; i < n; i++) {
        team[i].id = 100 + i;
        sprintf(team[i].name, "Emp%d", i + 1); /* team[i] works like an array */
    }
    for (int i = 0; i < n; i++)
        printf("%d %s\n", team[i].id, team[i].name);
    free(team);
    return 0;
}
```

Output:

```
Employee ID: 1, Name: Alice
100 Emp1
101 Emp2
102 Emp3
```

10.9 typedef with Structures

typedef gives a structure type a shorter name, so **struct** need not be repeated in every declaration.

```
#include <stdio.h>
#include <math.h>

typedef struct {
    int x;
    int y;
} Point;                                /* Point is now a type name */

double distance(Point a, Point b) {
    int dx = a.x - b.x, dy = a.y - b.y;
    return sqrt(dx * dx + dy * dy);
}

int main() {
    Point p1 = {0, 0};                  /* no 'struct' keyword needed */
    Point p2 = {3, 4};
    printf("Distance = %.1f\n", distance(p1, p2));
    return 0;
}
```

Output:

```
Distance = 5.0
```

Topic 11: Union

A **union** is declared with the **union** keyword and looks just like a structure. The critical difference is memory: **all members of a union share the same block of memory**, starting at the same address. The union is only as big as its **largest member**, and it can hold a valid value for **only one member at a time**. Writing to one member overwrites the bytes the others also use.

11.1 Declaring and Using a Union

```
#include <stdio.h>
#include <string.h>

union Data {
    int i;
    float f;
    char str[20];
};

int main() {
    union Data data;

    data.i = 10;
    printf("data.i = %d\n", data.i);           /* fine, i was written last */

    data.f = 220.5;
    printf("data.f = %.1f\n", data.f);         /* overwrites the bytes of i */
    printf("data.i = %d <- garbage: those bytes now hold the float 220.5\n", data.i); /* fine, f was written last */

    strcpy(data.str, "Hello");
    printf("data.str = %s\n", data.str);        /* overwrites f as well */

    printf("Same address for i and f? %d\n", (void *)&data.i == (void *)&data.f);
    printf("sizeof(union Data) = %zu\n", sizeof(data));
    return 0;
}
```

Output:

```
data.i = 10
data.f = 220.5
data.i = 1130135552 <- garbage: those bytes now hold the float 220.5
data.str = Hello
Same address for i and f? 1
sizeof(union Data) = 20
```

11.2 Structure vs Union: Memory Comparison

```
#include <stdio.h>

struct S { int i; float f; char str[20]; }; /* separate space for each member */
union U { int i; float f; char str[20]; }; /* one shared space */

int main() {
    printf("sizeof(struct S) = %zu (4 + 4 + 20 = 28)\n", sizeof(struct S));
    printf("sizeof(union U) = %zu (size of the largest member)\n", sizeof(union U));
    return 0;
}
```

Output:

```
sizeof(struct S) = 28 (4 + 4 + 20 = 28)
sizeof(union U) = 20 (size of the largest member)
```

Feature	Structure (struct)	Union (union)
Memory allocation	Separate memory for each member	One shared block for all members

Feature	Structure (<code>struct</code>)	Union (<code>union</code>)
Size	Greater than or equal to the sum of all member sizes (padding possible)	Equal to the size of the largest member
Value storage	All members hold values at the same time	Only one member holds a valid value at a time
Modification	Changing one member does not affect the others	Changing one member overwrites all the others
Member addresses	Each member has its own address	All members start at the same address
Typical use	Records where every field is needed together (Student, Book)	A value that can be one of several types, or saving memory

11.3 Where Unions Are Useful

- A variable that is only ever **one of several types** at a time, such as a value in an interpreter that may be an int, a float or a string (see the tagged union in Topic 12.5).
- Memory-constrained embedded systems**, where minimising memory matters more than storing every possible value simultaneously.
- Viewing the **same bytes in different ways**, for example inspecting the individual bytes of an integer.

```
#include <stdio.h>

union Bytes {
    unsigned int  value;
    unsigned char b[4];           /* the same 4 bytes, seen one at a time */
};

int main() {
    union Bytes u;
    u.value = 0x12345678;
    printf("value = 0x%X\n", u.value);
    for (int i = 0; i < 4; i++)
        printf("b[%d] = 0x%02X\n", i, u.b[i]);
    return 0;
}
```

Output:

```
value = 0x12345678
b[0] = 0x78
b[1] = 0x56
b[2] = 0x34
b[3] = 0x12
```

NOTE: The order above (78 56 34 12) is for a **little-endian** processor, the common case (x86, ARM). Byte order is machine-dependent, which is one reason such tricks are used carefully.

Topic 12: Enumerated Type (enum)

An **enum** is a user-defined type made of a set of **named integer constants**. Its purpose is to give meaningful names to numbers, so a program is readable and self-documenting: **wednesday** says far more than the bare number **3**, though they are the same value.

12.1 Default Numbering

The first name is **0**, the second **1**, the third **2**, and so on, increasing by 1.

```
#include <stdio.h>

enum Weekday { Sunday, Monday, Tuesday, Wednesday, Thursday, Friday, Saturday };

int main() {
    enum Weekday today = Wednesday;           /* holds the integer 3 */

    if (today == Wednesday)
        printf("It is the middle of the week! (Value: %d)\n", today);

    /* names can be printed with a helper array indexed by the enum value */
    const char *names[] = {"Sunday", "Monday", "Tuesday", "Wednesday",
                           "Thursday", "Friday", "Saturday"};
    printf("Today is %s\n", names[today]);

    for (enum Weekday d = Sunday; d <= Saturday; d++)    /* enums work in loops */
        printf("%d=%s ", d, names[d]);
    printf("\n");
    return 0;
}
```

Output:

```
It is the middle of the week! (Value: 3)
Today is Wednesday
0=Sunday 1=Monday 2=Tuesday 3=Wednesday 4=Thursday 5=Friday 6=Saturday
```

12.2 Assigning Your Own Values

Any name can be given an explicit integer. Names that follow **continue counting upward** from that value.

```
#include <stdio.h>

enum StatusCode { OK = 200, NOT_FOUND = 404, SERVER_ERROR = 500 };
enum Level      { LOW = 1, MEDIUM, HIGH = 10, CRITICAL };    /* 1, 2, 10, 11 */

int main() {
    printf("OK=%d NOT_FOUND=%d SERVER_ERROR=%d\n", OK, NOT_FOUND, SERVER_ERROR);
    printf("LOW=%d MEDIUM=%d HIGH=%d CRITICAL=%d\n", LOW, MEDIUM, HIGH, CRITICAL);
    return 0;
}
```

Output:

```
OK=200 NOT_FOUND=404 SERVER_ERROR=500
LOW=1 MEDIUM=2 HIGH=10 CRITICAL=11
```

12.3 enum with switch

Enums and **switch** fit together naturally: each **case** label is a readable name instead of a magic number.

```
#include <stdio.h>

enum Light { RED, YELLOW, GREEN };

void react(enum Light signal) {
    switch (signal) {
        case RED:    printf("Stop\n");        break;
        case YELLOW: printf("Get ready\n");    break;
        case GREEN:  printf("Go\n");          break;
        default:     printf("Invalid signal\n");
    }
}

int main() {
    react(RED);
    react(GREEN);
    react(YELLOW);
    return 0;
}
```

Output:

```
Stop
Go
Get ready
```

12.4 enum vs #define

Aspect	#define	enum
Grouping	Each constant is separate and unrelated	Related constants are grouped under one named type
Automatic values	No, each value is written by hand	Yes, 0, 1, 2, ... assigned automatically
Handled by	Preprocessor (plain text replacement)	Compiler (many compilers can warn about mixing unrelated enums)
Scope	From the definition to the end of the file	Follows normal block scope rules
Best for	Simple constants like PI	A fixed set of related options: days, directions, states, error codes

NOTE: In C, an enum constant is really an **int**, and the compiler does **not** stop you from storing an out-of-range number in an enum variable. The benefit is readability and (on many compilers) extra warnings, not strict protection.

12.5 Putting It Together: enum + struct + union (tagged union)

A **tagged union** stores a value that can be one of several types, together with an **enum tag** that records which member is currently valid. This is the pattern used by interpreters and parsers, and it shows why unions are safe only when you track which member was written last.

```
#include <stdio.h>
#include <string.h>

typedef enum { T_INT, T_FLOAT, T_TEXT } Type;          /* the tag */

typedef struct {
    Type type;                                         /* which member is valid */
    union {
        int i;
        float f;
        char text[20];
    } u;                                              /* shared storage */
} Value;

void show(Value v) {
    switch (v.type) {
        case T_INT: printf("int : %d\n", v.u.i); break;
        case T_FLOAT: printf("float : %.2f\n", v.u.f); break;
        case T_TEXT: printf("text : %s\n", v.u.text); break;
    }
}

int main() {
    Value a, b, c;
    a.type = T_INT; a.u.i = 42;
    b.type = T_FLOAT; b.u.f = 3.14f;
    c.type = T_TEXT; strcpy(c.u.text, "Hello");

    show(a);
    show(b);
    show(c);
    printf("sizeof(Value) = %zu (tag + one shared 20-byte area)\n", sizeof(Value));
    return 0;
}
```

Output:

```
int : 42
float : 3.14
text : Hello
sizeof(Value) = 24 (tag + one shared 20-byte area)
```

Topic 13: Quick Revision

13.1 Structure vs Union vs Enum at a Glance

Feature	struct	union	enum
Keyword	<code>struct</code>	<code>union</code>	<code>enum</code>
Holds	Members of different types	Members of different types	Named integer constants
Memory	Sum of members (plus padding)	Size of the largest member	Size of an <code>int</code>
Members valid at once	All	Only one	One value at a time
Member access	<code>.</code> or <code>-></code>	<code>.</code> or <code>-></code>	Used by name directly
Main purpose	Group related data into a record	Save memory / one-of-many types	Readable names for fixed choices

13.2 Array vs Pointer

Feature	Array	Pointer
Stores	A block of elements	One address
Size	<code>sizeof(arr)</code> = whole array	<code>sizeof(p)</code> = size of the address (8 bytes on 64-bit)
Can be reassigned / incremented	No (array name is a constant)	Yes (<code>p++</code> , <code>p = other</code>)
Memory	Allocated automatically at declaration	Must be pointed at something (variable or <code>malloc</code> block)
Passed to a function	Decays into a pointer to the first element	Address is copied

13.3 Key Points to Remember

- A function has a **declaration (prototype)**, a **definition** and a **call**; a function returns at most one value.
- Arguments are passed **by value**; passing an **address** gives call by reference. Arrays are shared with the function because the name decays to a pointer.
- Array indices start at **0**; C never checks bounds. 2D arrays are stored in **row-major** order.
- A string is a `char` array ending in `'\0'`; leave room for it. Prefer `fgets` over `gets`.
- `&` gives an address, `*` follows one. Pointer arithmetic is **scaled by the type size**. `arr[i]` is `*(arr + i)`.
- Set pointers to `NULL` when they point to nothing and right after `free()`.
- Every recursive function needs a **base case**; each call must move toward it.
- static** locals keep their value between calls; **auto** locals do not; **extern** shares a global across files.
- `malloc` = uninitialised, `calloc` = zeroed, `realloc` = resize, `free` = release exactly once. Always check for `NULL`.
- Never return the address of a local variable; that creates a **dangling pointer**.
- Structure members are reached with `.`; through a pointer use `->`. Arrays inside structures need `strcpy`, not `=`.
- Structures can be nested, stored in arrays, passed by value or pointer, and allocated dynamically.

- A union shares one block of memory; only the member written last is valid.
- An enum gives names to integers, starting at 0 unless you assign values.

13.4 Common Mistakes

Mistake	Why it is wrong	Fix
Missing <code>;</code> after a <code>struct</code> / <code>union</code> / <code>enum</code> declaration	The declaration is a statement and needs a terminator	<code>struct A { ... };</code>
<code>s = "text";</code> for a char array	Arrays cannot be assigned after declaration	<code>strcpy(s, "text");</code>
Using <code>gets()</code>	No size limit: buffer overflow	<code>fgets(s, sizeof s, stdin);</code>
Forgetting <code>&</code> in <code>scanf("%d", x)</code>	<code>scanf</code> needs the address to store into	<code>scanf("%d", &x);</code> (no <code>&</code> for <code>%s</code> on arrays)
Swapping with <code>void swap(int a, int b)</code>	Only copies are swapped	<code>void swap(int *a, int *b)</code> called with <code>swap(&x, &y)</code>
Using <code>sizeof(arr)</code> inside a function	<code>arr</code> is a pointer there	Pass the length as a parameter
Using memory after <code>free()</code>	Dangling pointer, undefined behaviour	<code>free(p); p = NULL;</code>
Not freeing allocated memory	Memory leak	One <code>free()</code> per successful allocation
Recursive function without a base case	Infinite recursion, stack overflow	Add a stopping condition
Forgetting <code>break</code> in <code>switch</code>	Execution falls through into the next case	End each case with <code>break;</code>
Comparing structures with <code>==</code>	Not allowed in C	Compare member by member
Reading a union member other than the last one written	Bytes belong to a different member	Track the active member (tagged union)

13.5 Coverage Map: Where the Old Units Went

Everything from the original Unit 3, Unit 4 and Unit 5 notes is kept; only the order has changed so that related ideas are together.

Original topic	Now found in
Unit 3: Introduction to functions, function parts	Topic 1.1 to 1.3
Unit 3: Types of user-defined functions	Topic 1.4
Unit 3: Library functions (string, math, I/O, ctype, stdlib)	Topic 2 (math, I/O, ctype, stdlib) and Topic 4.3 (string)
Unit 3: Memory allocation functions	Topic 9
Unit 3: Call by value / call by reference	Topic 6.2
Unit 3: Recursion	Topic 7
Unit 3: Storage classes	Topic 8
Unit 3: Passing and returning values	Topic 6.1, 6.3, 6.4
Unit 4: Arrays (1D, 2D, multi-dimensional), uses, matrices	Topic 3
Unit 4: String arrays	Topic 4.5
Unit 4: Pointers: address-of, dereference, pointer to variable / array, pointer arithmetic	Topic 5.1, 5.2
Unit 4: Array of pointers, pointer to pointer	Topic 5.3, 5.4
Unit 4: Pointer to structure, pointers within structure	Topic 10.5
Unit 4: Use of pointers in dynamic programming, dangling pointer	Topic 9
Unit 5: Structure declaration, initialisation	Topic 10.1, 10.2
Unit 5: Memory allocation for structures	Topic 10.8
Unit 5: Pointers, arrays, functions and nesting with structures	Topic 10.5, 10.4, 10.6, 10.7
Unit 5: Union and difference between structure and union	Topic 11
Unit 5: Enumerated type	Topic 12